Attention from First Principles

Document 1 of 6 — Building a GPT from Scratch

This document derives scaled dot-product attention from the ground up, builds multi-head attention on top of it, and introduces the causal mask needed for language modeling. By the end, every line of `CausalSelfAttention` in `src/model.py` should make sense.

---

1 What Problem Does Attention Solve?

Consider a sequence of T token embeddings, each a vector in $\mathbb{R}^d$. Every position in the sequence carries some meaning, but meaning in language is deeply contextual — the word "bank" means something different depending on whether "river" or "account" appears nearby. So each position needs a mechanism to gather information from other positions.

Before attention, the dominant approach was recurrent neural networks (RNNs). An RNN processes a sequence left to right, maintaining a hidden state that gets updated at each step. This has two problems:

1. **Sequential bottleneck.** You cannot compute the output at position t until you have computed positions 1 through $t-1$. This kills parallelism on GPUs.
2. **Information decay.** Information from position 1 must survive through every hidden-state update to reach position T. In practice, gradients vanish or explode, and the model struggles with long-range dependencies despite techniques like gating (LSTM, GRU).

Attention solves both problems with a single idea: **let each position compute a weighted average of all positions, where the weights are determined dynamically by the content at those positions.** There is no sequential dependency — every position can attend to every other position in parallel. And there is no distance decay — position 1 is just as accessible as position $T-1$, if the content says it should be.

The rest of this document is about making that idea precise.

---

2 Dot Products as Similarity

The mechanism that determines "how much should position i attend to position j?" is the dot product.

Recall from linear algebra: for two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$,

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos(\theta)$$

where θ is the angle between them. When $\mathbf{a}$ and $\mathbf{b}$ point in similar directions (small θ), $\cos(\theta)$ is close to 1 and the dot product is large. When they are orthogonal, the dot product is zero. When they point in opposite directions, it is negative.

In attention, we assign each position two roles:

- A **query** vector $\mathbf{q}$ — "what am I looking for?"
- A **key** vector $\mathbf{k}$ — "what do I contain?"

The dot product $\mathbf{q} \cdot \mathbf{k}$ measures how well the query at one position matches the key at another. High dot product means high relevance.

Concrete example. Suppose we have 3 positions with query and key vectors in $\mathbb{R}^4$:

```
import torch

Q = torch.tensor([
    [1.0,  0.0,  1.0,  0.0],  # query at position 0
    [0.0,  1.0,  0.0,  1.0],  # query at position 1
    [1.0,  1.0,  0.0,  0.0],  # query at position 2
])

K = torch.tensor([
    [1.0,  0.0,  1.0,  0.0],  # key at position 0
    [0.0,  1.0,  0.0,  1.0],  # key at position 1
    [1.0,  0.0,  0.0,  1.0],  # key at position 2
])

scores = torch.matmul(Q, K.transpose(-2, -1))
print(scores)
```

Output:

```
tensor([[2., 0., 1.],
        [0., 2., 1.],
        [1., 1., 1.]])
```

Reading row 0: position 0's query is most similar to position 0's key (score 2), completely orthogonal to position 1's key (score 0), and moderately similar to position 2's key (score 1). This is exactly what you would verify by hand — $\mathbf{q}_0 = [1, 0, 1, 0]$ and $\mathbf{k}_0 = [1, 0, 1, 0]$ share all their nonzero components, while $\mathbf{k}_1 = [0, 1, 0, 1]$ has nonzero components exactly where $\mathbf{q}_0$ is zero.

Dimensions. If Q is (T, d_k) and K is (T, d_k), then $QK^\top$ is $(T, d_k) \times (d_k, T) = (T, T)$. Each entry (i, j) in this matrix is the raw attention score from position i to position j. We get a score for every pair of positions.

---

3 Why Scale by $\sqrt{d_k}$

Those raw scores have a problem: their magnitude depends on d_k, and this matters a lot when we feed them into softmax.

Here is the variance argument. Suppose each component of $\mathbf{q}$ and $\mathbf{k}$ is drawn independently from $\mathcal{N}(0, 1)$. Then:

- Each product $q_i k_i$ has mean $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$ (since they are independent and zero-mean).
- Each product $q_i k_i$ has variance $\mathrm{Var}(q_i k_i) = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1 \cdot 1 = 1$.
- The dot product $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. Since they are independent, $\mathrm{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$.
- The standard deviation is $\sqrt{d_k}$.

So the dot product is a zero-mean random variable with standard deviation $\sqrt{d_k}$. When $d_k = 64$ (a common head dimension), the standard deviation is 8. Dot products routinely land in the range $[-16, +16]$ or wider.

Why does this matter? Consider what softmax does with large inputs. If one score is 10 and the rest are near 0, then $\exp(10) \approx 22026$ dominates the denominator, and the output is nearly one-hot. In the backward pass, the gradients of softmax near a one-hot output are very close to zero — the model is "confident" and stops learning. This is the softmax saturation problem.

The fix is to divide by $\sqrt{d_k}$ before applying softmax, normalizing the scores back to unit variance:

```
import math

d_k = Q.size(-1)
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
```

There is a nice way to think about this: dividing by $\sqrt{d_k}$ is equivalent to setting the softmax **temperature** to $\sqrt{d_k}$. Higher temperature makes the distribution more uniform (softer attention); lower temperature makes it more peaked. By scaling, we ensure the temperature is calibrated regardless of dimension, so the model starts training with reasonable gradient flow.

---

4 Softmax Turns Scores into Weights

After scaling, we have a (T, T) matrix of real-valued scores. We need to convert each row into a probability distribution — non-negative values that sum to 1 — so we can take a weighted average. Softmax does this:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Properties:

- Every output is in $(0, 1)$ — strictly positive, never exactly zero.
- The outputs sum to 1 across the dimension we apply softmax over.
- It is monotonic: larger inputs get larger probabilities.
- It is differentiable everywhere, which is necessary for backpropagation.

In code:

```
import torch.nn.functional as F

weights = F.softmax(scores, dim=-1)  # (T, T)
```

The `dim=-1` means we apply softmax across the last dimension (the key dimension), so each row independently becomes a probability distribution. Row i tells us: "how much should position i attend to each other position?"

Connection to temperature. More generally, softmax with temperature τ is $\text{softmax}(\mathbf{z}/\tau)$. When τ is small, the distribution sharpens toward a one-hot on the maximum element. When τ

is large, the distribution flattens toward uniform. The $\sqrt{d_k}$ scaling is exactly this: we implicitly set $\tau = \sqrt{d_k}$, preventing the distribution from being too sharp at initialization.

---

5 The Full Attention Formula

We now have all the ingredients. Each position also has a **value** vector $\mathbf{v}$ — "what I contribute to others if they attend to me." The full scaled dot-product attention formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Dimension walkthrough, step by step:

Expression	Shape	What it represents
Q	(T, d_k)	Query vectors for each position
K	(T, d_k)	Key vectors for each position
V	(T, d_v)	Value vectors for each position
$QK^\top$	(T, T)	Raw attention scores for all position pairs
$QK^\top/\sqrt{d_k}$	(T, T)	Scaled scores (unit variance)
$\text{softmax}(QK^\top/\sqrt{d_k})$	(T, T)	Attention weights (each row sums to 1)
$\text{softmax}(\cdots)\,V$	(T, d_v)	Weighted combination of value vectors

The output has shape (T, d_v). Each row i of the output is a weighted average of the value vectors, where the weights come from how well position i's query matches each position's key. Typically $d_v = d_k$, so the output has the same dimensionality as the input queries and keys.

Complete implementation:

```python
import math
import torch
import torch.nn.functional as F

def attention(Q, K, V, mask=None):
    """Scaled dot-product attention.

    Args:
        Q: (*, T, d_k) query vectors
        K: (*, S, d_k) key vectors
        V: (*, S, d_v) value vectors
        mask: broadcastable to (*, T, S), 0 where attention is
            forbidden

    Returns:
        (*, T, d_v) attended values
    """
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V)
```

A few things to note:

- The `*` in the shape annotations means this works with arbitrary leading batch dimensions. This is important for multi-head attention (Section 6).
- The `mask` argument is for causal masking (Section 7). Where the mask is 0, we set the score to $-\infty$ before softmax, which makes the weight exactly 0.
- The function is pure matrix multiplications and elementwise operations — extremely GPU-friendly.

---

6 Multi-Head Attention

A single set of Q, K, V projections computes a single attention pattern: one (T, T) matrix of weights. But "relevance" is not one-dimensional. Position j might be relevant to position i for syntactic reasons (it is the verb's subject), for semantic reasons (it discusses the same topic), for positional reasons (it is the previous word), and so on.

Multi-head attention addresses this by running multiple attention computations in parallel, each with its own learned projections, and then combining the results.

The math. Given input X of shape (T, d_{model}), and h heads:

For each head i (where $i = 1, \ldots, h$):

$$\text{head}_i = \text{Attention}(XW_i^Q,\ XW_i^K,\ XW_i^V)$$

where W_i^Q, W_i^K, W_i^V are each (d_{model}, d_k) projection matrices and $d_k = d_{\text{model}}/h$.

Then concatenate and project:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

where W^O is $(d_{\text{model}}, d_{\text{model}})$.

Each head operates in a d_k-dimensional subspace. Head 1 might learn to attend based on syntactic structure; head 2 might attend to nearby positions; head 3 might track coreference. The output projection W^O learns to combine these different views.

The efficient implementation trick. Naively, h heads require $3h$ separate matrix multiplications for the projections. We can do it with just one by concatenating all the projection matrices:

1. **Single combined projection.** Multiply X by a single weight matrix W_{qkv} of shape $(d_{\text{model}}, 3 \cdot d_{\text{model}})$ to produce all Q, K, V for all heads at once:

```
qkv = self.c_attn(x)  # (B, T, 3 * d_model)
```

2. **Split into Q, K, V**, each of shape (B, T, d_{model}):

```
q, k, v = qkv.split(self.n_embd, dim=2)
```

3. **Reshape into heads.** We reshape (B, T, d_{model}) into $(B, T, n_{\text{head}}, d_k)$ and then transpose to $(B, n_{\text{head}}, T, d_k)$:

```
q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
```

Now the tensor has shape $(B, n_{\text{head}}, T, d_k)$. The n_{head} dimension acts as an additional batch dimension — the (T, d_k) attention computation runs independently for each head.

4. **Run attention.** Standard scaled dot-product attention over the last two dimensions. Each head computes its own (T, T) weight matrix and its own (T, d_k) output.

5. **Reassemble.** Transpose back and reshape to merge heads:

```
y = y.transpose(1, 2).contiguous().view(B, T, d_model)
```

6. **Output projection.** A final linear layer mixes the concatenated head outputs:

```
y = self.c_proj(y)  # (B, T, d_model)
```

Parameter count. We have four weight matrices of shape $(d_{\text{model}}, d_{\text{model}})$: one combined QKV projection (actually $(d_{\text{model}}, 3 \cdot d_{\text{model}})$, so three matrices worth) and one output projection. Total: $4 \cdot d_{\text{model}}^2$ parameters (ignoring biases). For our BabyGPT with $d_{\text{model}} = 384$, that is $4 \times 384^2 = 589{,}824$ parameters per attention layer.

You can see this exact implementation in `src/model.py` in the `CausalSelfAttention` class. The combined QKV projection is `self.c_attn`, and the output projection is `self.c_proj`. The attention computation itself is delegated to PyTorch's `F.scaled_dot_product_attention`, which fuses the scaling, masking, softmax, and matrix multiply into a single efficient kernel (flash attention).

---

7 Causal (Autoregressive) Masking

Everything so far describes **bidirectional** attention: every position can attend to every other position. This is fine for tasks where you have the full input (e.g., classification, encoding). But for language modeling — predicting the next token given the previous ones — position t must not see positions $t+1, t+2, \ldots, T$. That would be cheating.

The solution is a **causal mask**: a lower-triangular matrix of ones.

```
1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1
```

Row t has ones in columns 0 through t and zeros elsewhere. Position 0 can only see itself. Position 1 can see positions 0 and 1. Position $T-1$ can see everything.

We apply this mask to the attention scores **before** softmax. Where the mask is 0 (future positions), we set the score to $-\infty$:

```
mask = torch.tril(torch.ones(T, T))  # lower-triangular
scores = scores.masked_fill(mask == 0, float('-inf'))
```

Why $-\infty$ and not 0? Because of how softmax works. If we set a score to 0, then $\exp(0) = 1$, which is a perfectly valid nonzero weight — the model would still attend to the future position. We need the weight to be exactly zero, and the only way to get softmax to output zero is to feed it $-\infty$: $\exp(-\infty) = 0$. The masked position contributes nothing to the weighted average.

After masking and softmax, the weight matrix looks like:

```
[1.00  0.00  0.00  0.00]
[0.45  0.55  0.00  0.00]
[0.20  0.30  0.50  0.00]
[0.10  0.25  0.35  0.30]
```

Each row sums to 1, and all entries above the diagonal are exactly 0. Every position attends only to itself and to the past.

In `src/model.py`, we do not construct the mask manually. Instead, we pass `is_causal=True` to PyTorch's `F.scaled_dot_product_attention`:

```
y = F.scaled_dot_product_attention(
    q, k, v,
    attn_mask=None,
    dropout_p=self.attn_dropout.p if self.training else 0.0,
    is_causal=True,
)
```

This flag tells PyTorch to apply the causal mask internally, which is more efficient — especially when using flash attention, which never materializes the full (T, T) attention matrix in memory.

---

8 Looking Ahead

We have now covered the core computational primitive of the transformer: scaled dot-product attention, extended to multiple heads, with causal masking for autoregressive modeling.

But attention alone is not a transformer. It is one sublayer in a larger block that also includes:

- **Residual connections** — the input to each sublayer is added to its output, enabling gradient flow through deep networks.
- **Layer normalization** — stabilizes training by normalizing activations.
- **A feed-forward network** — a position-wise MLP that gives each position capacity to process its attended representation.

The next document, *02 — The Transformer Block*, shows how these components wrap around attention to form the repeating unit of the transformer architecture.